TECHNICAL PROJECT DOCUMENTATION · MERN STACK

S

SpendWiser

MERN Stack Financial Management System

Complete Technical Documentation · All Pages & Workflows · Interview Reference

React 19 | TypeScript | Node.js | Express | MongoDB | Mongoose | Gemini AI | Recharts | JWT

KEY FEATURES

- Ø=Ü° Transaction Management
- Ø>Ý AI-Powered Insights
- Ø=Üñ Bank SMS Parsing
- Ø=ÜÈ Investment Plans
- Ø=ÜÊ Real-time Analytics
- Ø=Ý Secure Authentication

Generated: 16 June 2026

SpendWiser · MIT License · Built with React + Express + MongoDB + Gemini AI

1. Project Overview

SpendWiser is a full-stack MERN (MongoDB, Express, React, Node.js) financial management web application tailored for Indian users. It enables individuals to track income and expenses, manage multiple wallets, parse bank SMS messages into transactions automatically, receive AI-powered spending insights using Google Gemini, and view personalized investment recommendations based on live government interest rates.

The project is architected as a monorepo with a clear separation between the React/TypeScript frontend (client/) and an Express/MongoDB backend (backend/). It is deployment-ready on Vercel with a vercel.json bridge that routes frontend and API traffic seamlessly.

Project Goals

- Help users track every rupee in and out of their accounts
- Parse bank SMS messages automatically to extract transaction data
- Provide AI-driven financial advice powered by Google Gemini
- Show live government investment interest rates scraped from official sources
- Support multiple wallets: Cash, Card, Savings
- Allow full data export to CSV/Excel for offline records

---

2. Technology Stack — Deep Explanation

2.1 Frontend

- **React 19 + TypeScript:** UI framework with full static type safety. All component props, state, and API responses are typed via types.ts (Wallet, Transaction, User, Budget interfaces).
- **Vite:** Build tool replacing Create React App. Provides extremely fast Hot Module Replacement (HMR) during development and optimized ESModule-based production bundles.
- **Tailwind CSS:** Utility-first CSS with custom design tokens. Uses glass-panel, bg-mesh, animate-scan, animate-reveal custom classes for the futuristic dark-theme aesthetic.
- **Recharts:** React charting library. Used for the BarChart in the Dashboard (spending by category) and budget gauges.
- **Lucide Icons:** Lightweight, consistent SVG icon library. Used throughout for all UI icons (TrendingUp, LogOut, Plus, Sparkles, etc.).
- **Custom LiquidGauge component:** A bespoke animated SVG gauge showing budget usage as a liquid fill — built from scratch in LiquidGauge.tsx.

2.2 Backend

- **Node.js + Express:** REST API server. Handles all data operations and business logic. Entry point: backend/server.js.
- **MongoDB + Mongoose:** NoSQL database for storing users, transactions, bank details, and investment rates. Mongoose provides schema definitions, validation, and ODM methods (find, save, etc.).
- **JWT (JSON Web Tokens):** Stateless authentication. After login, the server issues a JWT which the frontend stores and sends in every API request Authorization header.
- **bcryptjs:** Password hashing library. Passwords are hashed with bcrypt (salt rounds = 10) before being stored in MongoDB — never stored in plain text.
- **node-cron:** Scheduled job runner. Runs performScraping() at midnight on the 1st of every 2nd month to refresh government interest rate data.
- **CORS:** Cross-Origin Resource Sharing middleware configured to only accept requests from the trusted frontend URL.
- **Rate Limiter:** Express middleware that limits API requests to prevent brute-force attacks on login and register

endpoints.

- **ExcelJS:** Generates Excel (.xlsx) transaction backup files downloadable from the API.

2.3 AI & Services

- **Google Gemini API (@google/genai):** Powers the Insights and AI Suggestions views. The app sends the user's transaction list to Gemini, which analyzes spending patterns and returns personalized advice in structured JSON format.
- **Web Scraper (node-cron + scraperService.js):** Scrapes official Indian government financial websites for live PPF, NPS, Sukanya Samriddhi, and FD interest rates. Data is cached in MongoDB (Rates collection) and served instantly.
- **SMS Webhook (Twilio optional):** A POST endpoint /api/sms/webhook that accepts incoming bank SMS messages from Twilio and automatically parses them into transactions.

3. System Architecture & Data Flow

3.1 Full-Stack Architecture

SpendWiser follows a classic 3-tier architecture: Presentation !’ Application Logic !’ Data Persistence.

```
TIER 1: PRESENTATION (Browser)
  React 19 + TypeScript + Vite (port 3001)
  Components: Dashboard, Ledger, Insights, AI Advisor, Investment Plans...
       !•  REST API calls (fetch + JWT Bearer token in Authorization header)

TIER 2: APPLICATION (Server)
  Node.js + Express (port 5000)
  Routes !’ Controllers !’ Mongoose Models
  Services: geminiService, scraperService, excelService
  Middleware: auth, rateLimiter, validate, CORS
       !•  Mongoose ODM queries

TIER 3: DATA (MongoDB)
  Collections: users, transactions, bankdetails, rates
  Backup: data.xlsx (ExcelJS auto-generated)

EXTERNAL:
  Google Gemini API !• geminiService.ts (frontend) / AI analysis
  Govt. Websites    !• scraperService.js (backend scraper, bi-monthly cron)
```

3.2 Authentication Flow

```
1. User fills Login Form (LoginPage.tsx)
2. AuthService.login(email, password) !’ POST /api/auth/login
3. Backend:
   a. Validates request body (Joi schema via validators/schemas.js)
   b. Applies rate limiter (loginLimiter — max 5 attempts per 15 min)
   c. Finds User in MongoDB by email
   d. bcrypt.compare(password, user.hashedPassword)
   e. If match !’ jwt.sign({ userId }) !’ returns { token, user }
4. Frontend: stores token in localStorage via AuthService
```

3.3 Transaction Data Flow

```
USER ADDS TRANSACTION (via TransactionModal or MessageParserModal):
  1. addTransaction(txData) called in App.tsx
  2. Optimistic update: new tx added to local React state immediately (UI feels instant)
  3. POST /api/transactions with JWT token
  4. Backend transactionController.create():
     a. Validates request
     b. Creates Transaction document (Mongoose)
     c. Saves to MongoDB
     d. Appends row to data.xlsx (ExcelJS backup)
  5. Wallet balance recalculated in React state (income !' +balance, expense !' -balance)
  6. Budget spending recalculated via useEffect (transactions !' budgets.spent)
  7. UI re-renders with live data

ON APP LOAD:
  1. fetchTransactions() !' GET /api/transactions (with JWT)
  2. Backend returns all user transactions from MongoDB
  3. React state updated !' all views refreshed
  4. Fallback: if backend unreachable, load from localStorage cache
```

3.4 AI Insights Flow

```
USER CLICKS "Get AI Suggestions" (AISuggestionsView.tsx):
  1. handleGenerateSuggestions() called in App.tsx
  2. getAISuggestions(transactions) from geminiService.ts (frontend service)
  3. Gemini SDK: GoogleGenAI.getGenerativeModel('gemini-pro')
  4. Prompt constructed with full transaction list as JSON
  5. Gemini API returns structured JSON with:
     - spending_summary
     - saving_tips (array)
     - investment_recommendations (array)
     - budget_optimizations (array)
     - financial_health_score (0-100)
  6. aiSuggestions state updated !' AISuggestionsView renders cards
```

3.5 SMS Parsing Flow

```
OPTION A — Manual Paste (MessageParserModal.tsx):
  1. User clicks "Decrypt SMS" button
  2. Pastes bank SMS text into the modal
  3. Frontend sends POST /api/sms/parse with raw SMS text
  4. Backend parseSMS() uses regex patterns to extract:
     - amount (e.g., "INR 5,000.00", "Rs. 500")
     - transaction type (debit !’ expense, credit !’ income)
     - merchant name, date
  5. Pre-filled TransactionModal opens for user to confirm and save

OPTION B — Twilio Webhook (automated):
  1. Bank SMS arrives on user's phone
  2. Twilio forwards it via POST /api/sms/webhook
  3. Server auto-parses and creates transaction in MongoDB
```

4. Database Schema (MongoDB + Mongoose)

4.1 User Model (backend/models/User.js)

```
Field         Type      Constraints              Description
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
id            String    unique, required         UUID identifier
email         String    unique, required,        User's email (auto-lowercased)
                        lowercase
password      String    required                 bcrypt hash (NEVER plaintext)
name          String    optional                 Display name
createdAt     Date      default: Date.now()      Account creation timestamp
updatedAt     Date      default: Date.now()      Last update timestamp

PRE-SAVE HOOK: if password modified !’ bcrypt.hash(password, salt=10)
METHOD: comparePassword(candidate) !’ bcrypt.compare() !’ boolean
```

4.2 Transaction Model (backend/models/Transaction.js)

```
Field         Type      Default         Description
% % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % % %
id            String    uuidv4()        Unique transaction ID
amount        Number    —               Amount in INR
type          String    —               'income' | 'expense'
category      String    —               Food, Transport, Health, etc.
description   String    —               Human-readable note
date          String    —               ISO date string
walletId      String    —               Links to wallet (Main/Cash/Savings)
currency      String    'INR'           Currency code
source        String    'manual'        'manual' | 'sms' | 'api'
smsFrom       String    —               Sender number if from SMS
createdAt     Date      Date.now()      Creation timestamp
updatedAt     Date      Date.now()      Last update timestamp
```

4.3 BankDetails Model (backend/models/BankDetails.js)

Stores user bank account information securely in MongoDB (encrypted at the document level). Account numbers are masked in UI — only last 4 digits shown.

4.4 Rates Model (backend/models/Rates.js)

Stores scraped government investment interest rates (PPF, NPS, FD, Sukanya Samriddhi). Updated bi-monthly via node-cron job. Served instantly from this cache — no live scraping on each request.

---

5. REST API Endpoints

Auth Endpoints (/api/auth)

Endpoint	Method	Description
POST /api/auth/register	POST	Register new user. Body: {name, email, password}. Validates with Joi, rate-limited (5/hr), hashes password, returns JWT.
POST /api/auth/login	POST	Authenticate user. Body: {email, password}. Rate-limited (10/hr). Returns JWT + user object.
GET /api/auth/user/:userId	GET	Get user profile (JWT required). Returns user object.
PUT /api/auth/user/:userId	PUT	Update user profile (JWT required). Body: {name, email}.

Transaction Endpoints (/api/transactions)

Endpoint	Method	Description
GET /api/transactions	GET	Fetch all transactions for logged-in user (JWT required). Returns array sorted newest first.
POST /api/transactions	POST	Create new transaction. Body: Transaction object. Saves to MongoDB + Excel backup.
PUT /api/transactions/:id	PUT	Update transaction by ID. Body: partial Transaction fields.
DELETE /api/transactions/:id	DELETE	Delete transaction by ID from MongoDB and Excel backup.
GET /api/transactions/download	GET	Download data.xlsx file (full transaction backup in Excel format).

Other Endpoints

Endpoint	Method	Description

GET /api/bank-details/:userId

GET

Get bank details for a user (JWT required).
Masked account numbers in response.

POST /api/bank-details

POST

Save/update bank details (JWT required).
Body: {bankName, accountNumber,
ifsc, ...}.

DELETE /api/bank-details/:userId

DELETE

Delete all bank details for user.

POST /api/sms/webhook

POST

Twilio SMS webhook. Receives raw SMS, parses, creates transaction automatically.

GET /api/investment-rates/scrape

GET

Returns cached government interest rates (PPF, FD, NPS, etc.) from MongoDB.

6. All Frontend Pages — Deep Explanation

SpendWiser uses a tab-based SPA (Single Page Application) pattern. There is no page routing in the traditional sense — instead, App.tsx maintains an activeTab state and conditionally renders different view components inside a persistent layout (Sidebar + header + main content area). The Sidebar controls navigation by setting the activeTab.

> **DESIGN THEME**
> SpendWiser uses a cyberpunk / neural-network dark theme. Dark backgrounds, indigo/emerald accents, glowing borders, glass panels, and sci-fi terminology (Neural Overview, Vault Repository, Ledger Feed, etc.). All animations use custom Tailwind CSS keyframes (animate-reveal, animate-scan, glass-card-hover).

Page 1: Login / Register (LoginPage.tsx)

Purpose

The entry gate to the application. Users must authenticate before accessing any data. This component is shown when App.tsx detects no active user session (user state is null).

Features & Working

- **Dual Mode:** Single component handles both Login and Register via isLoginMode boolean state. Toggled by clicking "Sign Up / Sign In" link.
- **Login Flow:** Calls AuthService.login(email, password) !' POST /api/auth/login !' JWT returned !' stored in localStorage !' user state set in App.tsx !' full app loads.
- **Register Flow:** Calls AuthService.register(name, email, password) !' POST /api/auth/register !' same JWT flow.
- **Error Handling:** API errors displayed in a rose-colored alert box. Loading state shows a spinner inside the submit button.
- **Rate Limiting UI:** If too many failed attempts, backend returns 429 (Too Many Requests) which is shown as an error.
- **Persistence:** AuthService.getCurrentUser() checks localStorage on every app start — if a valid session exists, the user skips the login page.

Technical Details

- **AuthService:** A service class (client/services/authService.ts) that wraps all auth API calls and manages token storage.
- **JWT Storage:** JWT is stored in localStorage under a key like "auth_token". Sent as Bearer token in all subsequent API calls.

Page 2: Dashboard — "Neural Overview" (Dashboard.tsx)

Purpose

The central command center. Gives the user an at-a-glance financial overview with 3 KPI metrics, budget gauges, a spending bar chart, and a live transaction feed.

Sections & Working

- **3 Stat Cards:** Neural Balance (income - expense), Total Inflow (sum of all income transactions), Total Outflow (sum of all expenses). All computed via useMemo on the transactions array for performance.
- **Neural Pulse (Budget Gauges):** Displays each budget category (Food ¹5000, Transport ¹3000, Entertainment ¹2000) as an animated LiquidGauge. The liquid level = (spent/limit × 100)%. Colors shift from indigo to rose as budget approaches limit.
- **Spending Landscape (Bar Chart):** Recharts BarChart showing spending grouped by category. Data computed by grouping expense transactions by category using a reduce() call. Colors cycle through COLORS array.
- **Live Feed (Mini Transaction List):** The 5-10 most recent transactions sorted by date descending. Each

shows description, date, and amount with green (income) or white (expense) coloring. Filterable via the search bar.

- **Export Ledger Button:** Generates a CSV file client-side using Blob + URL.createObjectURL. Downloads immediately without any server call.
- **Search Bar:** Sticky glass-panel search input. Filters transactions by description or category. Connected to onSearch prop which updates searchQuery in App.tsx — shared with LedgerView.

Technical Details

- **useMemo:** Stats and filtered transactions use useMemo([transactions, searchQuery]) to avoid recalculating on every render.
- **LiquidGauge:** Custom SVG component. Uses clip-path and animated rect to simulate a liquid filling animation tied to the percentage value.

Page 3: Vault Repository — "VaultsView.tsx"

Purpose

Shows the user's wallets (financial accounts). Three default wallets: Main Checking (Card), Cash, and Savings. Each shows its name, type, current balance, and currency.

Working

- **Data Source:** Wallet state in App.tsx. Initialized from localStorage (app_wallets_v2) on first load. Falls back to MOCK_WALLETS with ¹0 balance if no stored data.
- **Balance Calculation:** Wallet balance is updated locally when addTransaction() is called. Income adds to balance, expense subtracts. Formula: balance + (type === income ? +amount : -amount).
- **Color Coding:** Each wallet has a color property (indigo for card, emerald for cash, amber for savings). Used for visual differentiation.

Page 4: Ledger Feed — "LedgerView.tsx"

Purpose

Full chronological list of all transactions. The user's complete financial history. Supports search, filtering, and deletion.

Features

- **Search & Filter:** Uses the shared searchQuery from App.tsx to filter transactions in real-time.
- **Sorted List:** Transactions sorted newest first by date.
- **Delete:** Trash icon on each transaction calls deleteTransaction(id) !' optimistic removal from state !' DELETE /api/transactions/:id to MongoDB.
- **Source Badge:** Each transaction shows its source (manual, SMS, API) via a small badge.
- **Download Excel:** Button calls GET /api/transactions/download to download data.xlsx from the backend.

Page 5: Gemini Core — "InsightsView.tsx"

Purpose

AI-powered spending analysis. Uses Google Gemini to analyze the user's transaction history and generate a narrative spending report.

How It Works

- **Trigger:** User clicks "Generate Insights" button. Only active if transactions.length > 0.
- **API Call:** getSpendingInsights(transactions) in geminiService.ts sends transaction data to Gemini API.
- **Prompt Engineering:** Constructs a prompt like: "Analyze these transactions and provide spending insights: [JSON array of transactions]. Include: total spent per category, biggest expense, saving opportunities."
- **Response:** Gemini returns markdown text. Displayed as formatted text in the view.
- **Loading State:** isLoadingInsights shows a pulsing skeleton/spinner while waiting for Gemini response.

Page 6: AI Advisor — "AISuggestionsView.tsx"

Purpose

Structured AI financial advice in card format. While InsightsView gives a narrative, AISuggestionsView gives actionable, categorized suggestions.

How It Works

- **Trigger:** User clicks "Get AI Suggestions". Calls getAISuggestions(transactions).
- **Gemini Prompt:** Asks Gemini to return a structured JSON response with specific keys: spending_summary, saving_tips[], investment_recommendations[], budget_optimizations[], financial_health_score (0-100).
- **Response Parsing:** Frontend parses the JSON string from Gemini into an AISuggestionsResult typed object.
- **Display:** Renders colored cards for each category: green for saving tips, indigo for investment recs, amber for budget optimizations. Shows health score as a large radial number.
- **Error Handling:** If Gemini returns malformed JSON, error is caught and a friendly message is shown.

Page 7: Investment Plans — "InvestmentPlansView.tsx"

Purpose

Shows personalized investment recommendations based on the user's average monthly savings and live government interest rates scraped from official sources.

How It Works

- **Rate Fetching:** On component mount, calls GET /api/investment-rates/scrape. Backend returns cached rates from MongoDB (updated bi-monthly by cron job).
- **Plans Shown:** PPF (Public Provident Fund), NPS (National Pension System), Sukanya Samriddhi Yojana, Fixed Deposits, Mutual Funds.
- **Personalization:** Calculates user's average monthly savings from transactions. Suggests how much to invest in each instrument based on their savings capacity.
- **Live Rates:** Interest rates shown are real scraped data — not hardcoded. The scraper parses tables from official government financial websites.

Page 8: User Profile — "UserProfileView.tsx"

Purpose

User account management and bank details storage.

Sections

- **Profile Info:** Shows user name and email. Edit button to update via PUT /api/auth/user/:userId.
- **Bank Details:** Form to enter bank name, account number, IFSC code, branch. Saved to MongoDB via POST /api/bank-details. Account numbers are masked in UI (last 4 digits only visible). Multiple banks supported.
- **Connected Accounts:** Shows all saved bank accounts as cards with masked numbers.

Page 9: Settings — "SettingsView.tsx"

Purpose

App-wide configuration: wallet management, budget configuration, and data export/import.

Sections

- **Wallet Management:** Add/edit/delete wallets. Changes persisted to localStorage (app_wallets_v2).
- **Budget Configuration:** Set monthly spending limits per category (Food, Transport, Entertainment). Live updates the Dashboard gauges.
- **Currency Settings:** Toggle between INR and other currencies (UI-level).
- **Data Export:** Download all transactions as JSON backup file using JSON.stringify + Blob download.

- **Data Import:** Read JSON file, parse, and restore transactions to state.
- **Theme / Display:** App-level display preferences.

Modals (Global Overlays)

TransactionModal.tsx — "Initialize Log"

- **Trigger:** Top-right "Initialize Log" button. Opens a full-screen modal overlay.
- **Fields:** Amount (¹), Type (Income/Expense toggle), Category (dropdown from Category enum), Description (text), Date (date picker), Wallet (dropdown from wallets).
- **Validation:** All fields required. Amount must be a positive number.
- **Save:** Calls onSave(txData) prop !’ addTransaction() in App.tsx !’ optimistic update + API call.

MessageParserModal.tsx — "Decrypt SMS"

- **Trigger:** "Decrypt SMS" button in the header.
- **How it works:** User pastes a raw bank SMS. Frontend sends to /api/sms/parse. Backend regex parses amount, type, merchant. Returns pre-filled transaction data. User confirms and saves.
- **Sample SMS patterns:** "INR 5,000.00 debited from A/c XXXX1234 on 15-Jun. Merchant: Swiggy."

SMSIntegrationModal.tsx — "SMS Setup"

- **Purpose:** Guide to configure Twilio so bank SMS messages are forwarded automatically to the webhook endpoint.
- **Shows:** Twilio setup instructions, webhook URL, and configuration steps.

7. Security Architecture

7.1 Authentication & Authorization

- **JWT (JSON Web Token):** Stateless auth. Server signs a JWT with a secret key. Client sends it in every request header. Server verifies signature on protected routes without hitting DB.
- **bcryptjs Password Hashing:** Passwords hashed with bcrypt (salt rounds = 10) before storage. Even if DB is breached, passwords cannot be reversed. Uses Mongoose pre-save hook so hashing is automatic.
- **Auth Middleware:** middleware/auth.js verifies JWT on every protected route. If invalid/expired !' 401 Unauthorized response.
- **Joi Validation:** Request body validated via validators/schemas.js using Joi before reaching controllers. Prevents malformed data from entering the system.

7.2 Rate Limiting

- **express-rate-limit:** Applied globally to /api/ routes and specifically stricter limits on /login and /register to prevent brute-force and credential stuffing attacks.
- **Login Limiter:** 5 attempts per 15 minutes per IP. Returns 429 Too Many Requests if exceeded.

7.3 Data Security

- **Bank Account Masking:** Account numbers stored in full in MongoDB but only last 4 digits are shown in the UI.
- **CORS Configuration:** Only the configured FRONTEND_URL origin is allowed. All other origins are blocked.
- **Environment Variables:** .env files store secrets (MONGODB_URI, JWT_SECRET, GEMINI_API_KEY). Never committed to Git (.gitignore).
- **HTTPS in Production:** Deployed on Vercel with automatic SSL/TLS.

---

8. Complete User Workflows

Workflow 1: New User Registration & First Transaction

```
1. Open app !' LoginPage shown (no session in localStorage)
2. Click "Need new credentials? [Sign Up]"
3. Enter Name + Email + Password !' Submit
4. POST /api/auth/register !' User saved to MongoDB (hashed password)
5. JWT returned !' stored in localStorage !' App loads
6. Dashboard shows (all zeros — no transactions yet)
7. Click "Initialize Log" (green button, top right)
8. TransactionModal opens
9. Fill:  ¹50,000 | Income | Salary | Monthly Salary | Main Checking
10. Click Save
11. Transaction appears in Dashboard Live Feed instantly (optimistic)
12. POST /api/transactions !' saved to MongoDB
13. Wallet balance updates: Main Checking + ¹50,000
14. Budget spending recalculated via useEffect
```

Workflow 2: SMS Auto-Parsing

```
1. Receive bank SMS: "INR 1,200.00 debited from A/c XXXX5678.
   Merchant: Zomato. Available balance: INR 48,800.00."
2. Click "Decrypt SMS" in header
3. MessageParserModal opens
4. Paste the SMS text
5. POST /api/sms/parse !’ backend regex extracts:
   amount = 1200, type = expense, description = "Zomato"
6. TransactionModal pre-filled with parsed data
7. User selects Category: Food, Wallet: Main Checking
8. Confirm !’ transaction saved
```

Workflow 3: Get AI Financial Advice

```
1. Navigate to "AI Advisor" (sidebar)
2. AISuggestionsView shown
3. Click "Get AI Suggestions"
4. Loading state shown
5. getAISuggestions(transactions) !’ Google Gemini API
6. Gemini analyzes all transactions, identifies patterns:
   - Zomato orders 15 times this month (Food overspend)
   - No savings transactions
   - Transport spend very low
7. Returns JSON:
   {
     financial_health_score: 62,
     saving_tips: ["Reduce food delivery by 30% to save  ¹3,600/month"],
     investment_recommendations: ["Start SIP with  ¹2,000/month in index funds"],
     budget_optimizations: ["Food budget exceeded by  ¹1,800"]
   }
8. Cards rendered in view with color-coded sections
```

Workflow 4: View Investment Plans

```
1. Navigate to "Investment Plans" (sidebar)
2. InvestmentPlansView mounts
3. GET /api/investment-rates/scrape !’ returns cached rates
4. App calculates: average_monthly_savings = total_income - total_expense / months
5. Investment cards shown:
   PPF: 7.1% p.a. | Recommended SIP:  ¹2,000/month
   NPS: 8-10% (market-linked) | Recommended:  ¹500/month
   FD: 7.5% p.a. | Lump-sum:  ¹10,000
   Sukanya Samriddhi: 8.2% (if applicable)
6. Rates come from government websites scraped by scraperService.js
   (updated every 2 months via cron job)
```

---

9. Project File Structure

```
spendwiser/
% % %  client/                          !• React + TypeScript frontend
%    % % %  App.tsx                      !• Root: auth state, tab routing, data fetching
%    % % %  types.ts                    !• TypeScript interfaces (Transaction, Wallet, User, Budget)
%    % % %  index.css                    !• Custom CSS (glass-panel, bg-mesh, animations)
%    % % %  services/
%    %    % % %  geminiService.ts        !• Google Gemini AI API calls
%    %    % % %  authService.ts          !• JWT management, login/register/logout
%    % % %  components/
%        % % %  LoginPage.tsx            !• Auth entry (login + register)
%        % % %  Sidebar.tsx              !• Navigation menu (tab switcher)
%        % % %  Dashboard.tsx            !• Neural Overview: stats, charts, live feed
%        % % %  VaultsView.tsx           !• Wallet cards
%        % % %  LedgerView.tsx           !• Full transaction history + delete
%        % % %  InsightsView.tsx         !• Gemini narrative spending report
%        % % %  AISuggestionsView.tsx    !• Structured AI advice cards
%        % % %  InvestmentPlansView.tsx  !• Govt-rate-based investment recs
%        % % %  UserProfileView.tsx      !• Profile + bank details management
%        % % %  SettingsView.tsx         !• Wallets, budgets, data export/import
%        % % %  TransactionModal.tsx     !• Add transaction form (modal overlay)
%        % % %  MessageParserModal.tsx   !• SMS paste-and-parse modal
%        % % %  SMSIntegrationModal.tsx  !• Twilio setup guide
%        % % %  LiquidGauge.tsx          !• Custom animated SVG budget gauge

% % %  backend/                         !• Node.js + Express backend
%    % % %  server.js                   !• Entry point: middleware, routes, DB connection
%    % % %  models/
%    %    % % %  User.js                !• Mongoose schema: user accounts
%    %    % % %  Transaction.js         !• Mongoose schema: financial transactions
%    %    % % %  BankDetails.js         !• Mongoose schema: bank account info
```

10. Key Technical Topics for Interview

10.1 What is MERN Stack?

MERN = MongoDB + Express + React + Node.js. A full-stack JavaScript ecosystem where the same language (JS/TS) is used on both frontend and backend. In SpendWiser: React (frontend UI), Express on Node.js (REST API server), MongoDB (NoSQL database).

10.2 How does JWT Authentication work?

JWT = JSON Web Token. A signed token that proves identity without server-side sessions.

```
1. User logs in !’ server verifies password !’ creates JWT:
   jwt.sign({ userId: "abc123" }, JWT_SECRET, { expiresIn: "7d" })
   JWT = base64(header) + "." + base64(payload) + "." + signature
2. Client stores JWT !’ sends in every request: Authorization: Bearer <JWT>
3. Server verifies: jwt.verify(token, JWT_SECRET) !’ trusts the userId in payload
4. No database hit for auth check — stateless and scalable
```

10.3 Why MongoDB over SQL here?

MongoDB is schema-flexible, which suits financial data that may evolve (different SMS formats, new transaction fields). It stores data as JSON-like documents, making it natural with JavaScript. Mongoose adds schema validation, type casting, and middleware hooks (like auto-hashing passwords).

10.4 How does Google Gemini AI work in this app?

The frontend uses @google/genai SDK to send structured prompts to Gemini Pro model. The prompt includes transaction JSON data. Gemini analyzes patterns and returns financial advice — either as markdown (InsightsView) or structured JSON (AISuggestionsView). The JSON parsing is done client-side with error handling for malformed responses.

10.5 What is Mongoose and why use it?

Mongoose is an ODM (Object Document Mapper) for MongoDB in Node.js. It provides: schema definitions with type validation, middleware hooks (pre-save for password hashing), model methods (comparePassword), and a clean API (Transaction.find(), transaction.save(), etc.).

10.6 What is CORS and why is it needed?

CORS = Cross-Origin Resource Sharing. Browsers block requests from one origin (localhost:3001) to a different origin (localhost:5000) by default. The Express CORS middleware adds the right HTTP headers to allow the frontend to communicate with the backend.

10.7 What is Optimistic UI Update?

When a user adds a transaction, SpendWiser updates the React state immediately (optimistic) before the API call completes. This makes the UI feel instant. If the API call fails, the state is rolled back. Code: setTransactions(prev => [newTx, ...prev]) runs before the fetch() call resolves.

10.8 How does the Web Scraper work?

scraperService.js uses Node.js HTTP to fetch pages from government websites. It parses HTML responses using string manipulation or cheerio to extract interest rate tables. Data is saved to MongoDB Rates collection. A node-cron job runs performScraping() at midnight on the 1st of every 2nd month. During API requests, getCachedRates() returns the in-memory cached version — no live scraping on every request.

10.9 How does the SMS Parser work?

The SMS parser uses regular expressions (regex) to extract financial data from bank SMS text. Common

patterns: amount (INR 1,200.00 or Rs. 500), debit/credit keyword, merchant name after specific keywords. Different banks have different SMS formats — the parser handles multiple patterns.

10.10 What is Rate Limiting and why?

express-rate-limit middleware tracks how many requests come from each IP address. For login/register routes, it limits to 5-10 attempts per 15 minutes. If exceeded, returns HTTP 429. Prevents brute-force password attacks and protects server resources.

---

11. Quick Interview Summary Points

WHAT DID YOU BUILD?

SpendWiser — a full-stack MERN financial management app with JWT auth, MongoDB database, AI-powered insights via Google Gemini, automated bank SMS parsing, live government investment rates via web scraping, and multi-wallet tracking.

WHAT'S TECHNICALLY COMPLEX?

Three things: (1) The web scraper that fetches live government interest rates on a cron schedule. (2) The SMS parser using regex to extract transaction data from unstructured bank SMS text. (3) Structured prompt engineering with Gemini to get typed JSON responses for AI suggestions.

HOW IS STATE MANAGED?

React useState in App.tsx — the root component is the single source of truth. Props are passed down to child views. Wallet and budget state persists via localStorage. Transactions are fetched from MongoDB and also cached in localStorage as fallback.

HOW IS SECURITY HANDLED?

bcrypt password hashing, JWT stateless authentication, Joi request validation, express-rate-limiting on auth routes, CORS whitelist, environment variable secrets, and account number masking in UI.

SpendWiser

MERN Stack Financial Management System

React 19 · TypeScript · Node.js · Express · MongoDB · Mongoose · JWT · Google Gemini AI

© 2026 SpendWiser · MIT License